

Disconnected Entity Graph in Entity Framework Core (EF Core)

In this article, I will discuss **Disconnected Entity Graph in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Batch Processing with a Job Scheduler](#) with Examples.

1. [What is a Disconnected Entity Graph in Entity Framework Core](#)
2. [How to Attach a Disconnected Entity Graph in Entity Framework Core?](#)
3. [Different Approaches to Attach a Disconnected Entity Graph in EF Core?](#)
4. [Attach\(\) Method in Entity Framework Core](#)
5. [Entry\(\) Method in Entity Framework Core](#)
6. [Add\(\) Method in Entity Framework Core](#)
7. [ChangeTracker.TrackGraph\(\) in Entity Framework Core](#)
8. [When to use Disconnected Entity Graph in Entity Framework Core?](#)

What is a Disconnected Entity Graph in Entity Framework Core?

An Entity Graph in EF Core refers to a graph-like structure representing a set of related entities and their relationships within your application's data model. That means an entity graph in Entity Framework Core refers to an entity and its related data. Entity Graphs represent and manage complex data relationships in your application.

Handling disconnected entity graphs in Entity Framework Core involves working with complex objects that include related entities. In a disconnected scenario, such as a Web application or an API, the entities are retrieved in one operation (e.g., HTTP request) and then sent back to the server in a modified state in a subsequent operation. Managing these disconnected entity graphs correctly is essential for maintaining data integrity and correctly persisting the changes to the database.

How to Attach a Disconnected Entity Graph in Entity Framework Core

In Entity Framework Core (EF Core), we can attach a disconnected entity graph to the context object using the **Attach**, **Add**, and **Entry** methods. Let us understand How to Attach a Disconnected Entity Graph in Entity Framework Core with an Example. Before that, let us first understand what is an Entity Graph in Entity Framework Core. Let us first create the Parent and Child Entities:

Student.cs

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int StandardId { get; set; }
        public virtual Standard Standard { get; set; }
        public virtual StudentAddress StudentAddress { get; set; }
    }
}
```

Standard.cs

```
namespace EFCoreCodeFirstDemo.Entities
```

```

{
    public class Standard
    {
        public int StandardId { get; set; }
        public string StandardName { get; set; }
        public string Description { get; set; }
        public ICollection<Student> Students { get; set; }
    }
}

```

StudentAddress.cs

```

using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public class StudentAddress
    {
        [Key]
        public int StudentId { get; set; } // PK and FK
        public string Address1 { get; set; }
        public string Address2 { get; set; }
        public Student Student { get; set; }
    }
}

```

In this case, the Student Entity is the Main or Parent entity. Standard, StudentAddress, and Courses are the Child Entities or related entities, and combined together, we can say it's an Entity Graph. Next, modify the context class as follows:

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
        }
        public DbSet<Student> Students { get; set; }
        public DbSet<Standard> Standards { get; set; }
        public DbSet<StudentAddress> StudentAddresses { get; set; }
    }
}

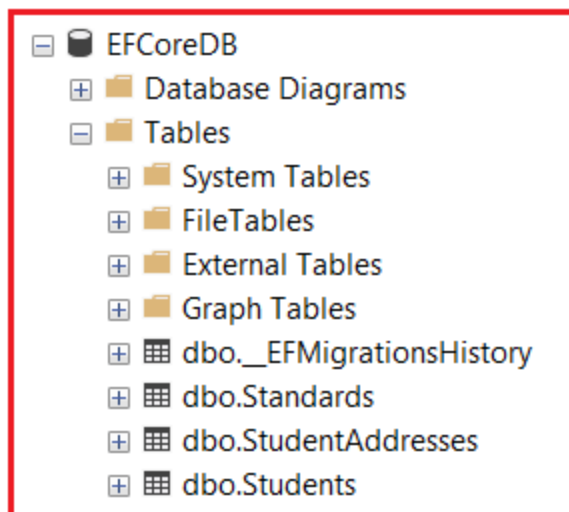
```

With the above changes, open the Package Manager Console and Execute the **add-migration** and **update-database** commands as follows. You can give any name to your migration. Here, I am giving **EFCoreDBMig1**. The

name that you are giving it should not be given earlier.

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231015090150_EFCoreDBMig1'.
Done.
```

Now, if you verify the database, then you will see the following:



Different Approaches to Attach a Disconnected Entity Graph in Entity Framework Core?

Entity Framework Core provides the following methods, which not only attach an entity to the context object but also change the EntityState of each entity in a disconnected entity graph:

- **Attach()**
- **Entry()**
- **Add()**

Let's see how the above methods change the EntityState of each entity in a disconnected entity graph in Entity Framework Core. Before Proceeding further, please Insert the following record into the Standards database table:

```
INSERT INTO Standards Values ('STD1', 'STD1 Description');
```

Attach() Method in Entity Framework Core (EF Core):

In Entity Framework Core, the Attach method attaches an entity to the DbContext instance so that the context object tracks it. Attaching an entity means EF Core is aware of its existence and will start tracking changes to that entity. This

is commonly used when working with entities disconnected from the context, such as entities created outside or fetched from another context object.

The **DbContext.Attach()** and **DbSet.Attach()** method attaches the specified disconnected entity graph and starts tracking it. The Attach Method attaches an entire Entity Graph (Parent and Child Entities) to the context object with the specified state to the parent entity. Also, it sets different Entity State to related entities, i.e., to the child entities.

The Attach() method sets Added EntityState to the root entity (in this case, Student) irrespective of whether it contains the Key value. A child entity containing the key value will be marked as Unchanged. Otherwise, it will be marked as Added. The output of the above example shows that the Student entity has an Added Entity State, the child entities with non-empty key values have an Unchanged Entity State, and the ones with empty key values have an Added state. The following table lists the behavior of the Attach() method when setting a different EntityState to a disconnected entity graph.

Attach()	Root entity with Key value	Root Entity with Empty or CLR default value	Child Entity with Key value	Child Entity with empty or CLR default value
context.Attach(entityGraph).State = EntityState.Added	Added	Added	Unchanged	Added
context.Attach(entityGraph).State = EntityState.Modified	Modified	Exception	Unchanged	Added
context.Attach(entityGraph).State = EntityState.Deleted	Deleted	Exception	Unchanged	Added

To better understand how to use the Attach method to attach an Entity Graph to the context object, please modify the Program class as follows. The following example code is self-explained, so please go through the comment lines.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                //Creating the Disconnected Entity Graph
                //Student Entity Graph (Student plus Standard, and
                StudentAddress)
                //Student is the Main Entity
                //Standard, and StudentAddress are the Child Entities
                var student = new Student()
                {
                    //Root Entity with Empty key
                    FirstName = "Pranaya",
                    LastName = "Rout",
                    StandardId = 1,
```

```

//Make Sure the StandardId with Value 1 Exists in the
Database, else you will get Exception
Standard = new Standard() //Child Entity with key value
{
    StandardId = 1,
    StandardName = "STD1",
    Description = "STD1 Description"
},
StudentAddress = new StudentAddress() //Child Entity with
Empty Key
{
    Address1 = "Address Line1",
    Address2 = "Address Line2"
}
};
//Creating an Instance of the Context class
using var context = new EFCoreDbContext();
//Attaching the Disconnected Student Entity Graph to the
Context Object
context.Attach(student).State = EntityState.Added;
//Checking the Entity State of Each Entity of student Entity
Graph
foreach (var entity in context.ChangeTracker.Entries())
{
    Console.WriteLine($"Entity:
{entity.Entity.GetType().Name}, State: {entity.State} ");
}
// Save changes to persist the changes to the database
context.SaveChanges();
Console.WriteLine("Entity Graph Saved Successfully");
Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); ;
}
}
}
}

```

Output:

```

Entity: Student, State: Added
Entity: StudentAddress, State: Added
Entity: Standard, State: Unchanged
Entity Graph Saved Successfully

```

In the above example, the **student** is an instance of the Student entity graph, which includes references to StudentAddress and Standard entities. **context.Attach(student).State = EntityState.Added;** attaches the **student** entity graph to a context object and sets Added state.

Entry() Method in Entity Framework Core:

In Entity Framework Core (EF Core), the Entry method is used to get an EntityEntry object representing an entity that the DbContext tracks. The EntityEntry object provides various methods and properties that allow us to inspect and manipulate the state of the entity being tracked, such as changing property values, setting the entity state, and more.

Consider the following example for a better understanding. In this case, only the root entity will be added, updated, or modified in the database. It will not affect the Child Entities.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                //Creating the Disconnected Entity Graph
                //Student Entity Graph (Student plus Standard, and
                StudentAddress)
                //Student is the Main Entity
                //Standard, and StudentAddress are the Child Entities
                var student = new Student()
                {
                    //Root Entity with Empty key
                    FirstName = "Pranaya",
                    LastName = "Rout",
                    StandardId = 1,
                    Standard = new Standard() //Child Entity with key value
                    {
                        StandardId = 1,
                        StandardName = "STD1",
                        Description = "STD1 Description"
                    },
                    StudentAddress = new StudentAddress() //Child Entity with
                    Empty Key
                    {
                        Address1 = "Address Line1",
                        Address2 = "Address Line2"
                    }
                };
                //Creating an Instance of the Context class
                using var context = new EFCoreDbContext();
                //Attaching the Disconnected Student Entity Graph to the
                Context Object
                context.Entry(student).State = EntityState.Added;
                //Checking the Entity State of Each Entity of student Entity
                Graph
                foreach (var entity in context.ChangeTracker.Entries())
```

```

        {
            Console.WriteLine($"Entity:
{entity.Entity.GetType().Name}, State: {entity.State} ");
        }
        // Save changes to persist the changes to the database
        context.SaveChanges();
        Console.Read();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}"); ;
    }
}
}
}

```

Output: Entity: Student, State: Added

In the above example, **context.Entry(student).State = EntityState.Added;** attaches an entity to a context and applies the specified EntityState (in this case, Added) to the root entity, irrespective of whether it contains a Key property value. It ignores all the child entities in a graph and does not attach or set their EntityState. The following table lists different behaviors of the DbContext.Entry() method.

Set EntityState using Entry()	Root entity with Key value	Root Entity with Empty or CLR default value	Child Entities with/out Key value
context.Entry(entityGraph).State = EntityState.Added	Added	Added	Ignored
context.Entry(entityGraph).State = EntityState.Modified	Modified	Modified	Ignored
context.Entry(entityGraph).State = EntityState.Deleted	Deleted	Deleted	Ignored

Add() Method in Entity Framework Core (EF Core):

The DbContext.Add and DbSet.Add methods attach an entity graph to a context and set Added EntityState to a root and child entity regardless of whether a key value exists. This means that the entity is considered new and should be inserted into the database when we call the SaveChanges method. The Add method is commonly used when creating and inserting a new record into the database. In this case, providing explicit values for the Identity column is restricted. You will get a Run Time Exception if you provide the Identity Column values. For a better understanding, please have a look at the following example:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try

```

```

{
    //Creating the Disconnected Entity Graph
    //Student Entity Graph (Student plus Standard, and
StudentAddress)
    //Student is the Main Entity
    //Standard, and StudentAddress are the Child Entities
    var student = new Student()
    {
        //Root Entity with key
        //StudentId = 1, //It is Identity, so you cannot set
Explicit Value
        FirstName = "Pranaya",
        LastName = "Rout",
        StandardId = 1,
        Standard = new Standard() //Child Entity with key value
        {
            // StandardId = 1, //It is Identity, so you cannot
set Explicit Value
            StandardName = "STD1",
            Description = "STD1 Description"
        },
        StudentAddress = new StudentAddress() //Child Entity with
Empty Key
        {
            Address1 = "Address Line1",
            Address2 = "Address Line2"
        }
    };
    //Creating an Instance of the Context class
    using var context = new EFCoreDbContext();
    //Attaching the Disconnected Student Entity Graph to the
Context Object
    context.Students.Add(student);
    //Checking the Entity State of Each Entity of student Entity
Graph
    foreach (var entity in context.ChangeTracker.Entries())
    {
        Console.WriteLine($"Entity:
{entity.Entity.GetType().Name}, State: {entity.State} ");
    }
    // Save changes to persist the changes to the database
    context.SaveChanges();
    Console.WriteLine("Entity Graph Added");
    Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); ;
}
}
}

```

```
}
```

Output:

```
Entity: Student, State: Added
Entity: Standard, State: Added
Entity: StudentAddress, State: Added
Entity Graph Added
```

The following diagram lists the possible EntityState of each entity in a graph using the DbContext.Add or DbSet.Add methods.

Method	Root entity with/out Key value	Child Entities with/out Key value
DbContext.Add(entityGraph) DbSet.Add(entityGraph)	or Added	Added

ChangeTracker.TrackGraph() in Entity Framework Core (EF Core)

The TrackGraph (a static method in the ChangeTracker class) method allows us to manually set the entity state of an entire graph or set the state based on some condition, which is then tracked by the context object. This is useful when working with a complex object graph and managing the state of entities within that graph, such as marking some entities as added, modified, or deleted and ensuring that EF Core tracks those changes correctly. The following is the signature of the TrackGraph method.

Signature: `public virtual void TrackGraph(object rootEntity, Action<EntityEntryGraphNode> callback)`

The ChangeTracker TrackGraph() method tracks an entity and any reachable entities by traversing its navigation properties. The specified callback is called for each reachable entity, and an appropriate EntityState is set for each entity. The callback function allows us to implement a custom logic to set the appropriate state. If no state is set, the entity remains untracked. The following example demonstrates the ChangeTracker TrackGraph method.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                //Creating the Disconnected Entity Graph
                //Student Entity Graph (Student plus Standard, and
                StudentAddress)
                //Student is the Main Entity
                //Standard, and StudentAddress are the Child Entities
                var student = new Student()
```

```

{
    //Root Entity without key
    FirstName = "Pranaya",
    LastName = "Rout",
    StandardId = 1,
    Standard = new Standard()    //Child Entity with key value
    {
        StandardId = 1,
        StandardName = "STD1",
        Description = "STD1 Description"
    },
    StudentAddress = new StudentAddress() //Child Entity with
Empty Key
    {
        Address1 = "Address Line1",
        Address2 = "Address Line2"
    }
};
//Creating an Instance of the Context class
using var context = new EFCoreDbContext();
context.ChangeTracker.TrackGraph(student, e =>
{
    if (e.Entry.IsKeySet)
    {
        //If Key is Available set the State as Unchanged or
Modified as Per Your Requirement
        e.Entry.State = EntityState.Unchanged;
    }
    else
    {
        // If Key is not Available set the State as Added
        e.Entry.State = EntityState.Added;
    }
});
//Checking the Entity State of Each Entity of student Entity
Graph
foreach (var entity in context.ChangeTracker.Entries())
{
    Console.WriteLine($"Entity:
{entity.Entity.GetType().Name}, State: {entity.State} ");
}
// Save changes to persist the changes to the database
context.SaveChanges();
Console.WriteLine("Entity Graph Saved..");
Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); ;
}
}

```

```
}  
}
```

Output:

```
Entity: Student, State: Added  
Entity: StudentAddress, State: Added  
Entity: Standard, State: Unchanged  
Entity Graph Saved..
```

In the above example, the `TrackGraph()` method sets the state for each entity of the Student entity graph. The first parameter is an entity graph, and the second parameter is a function that sets the state of each entity. We used a lambda expression to set the Unchanged state for entities that have valid key values and the Added state for entities that have empty key values. The `IsKeySet` becomes true when an entity has a valid key property value. So, we can use the `ChangeTracker TrackGraph()` method to set different States for each entity in a graph.

Another Example to Understand TrackGraph() Method:

Suppose we want to set the Root Entity State as Modified when the Key Property Value is available; if the key value is unavailable, we need to set the state as Added. But for the Child Entity, we need to set the Entity State as UnChanged when the Key Value is available, the Entity State should be Added. To achieve this, please modify the Main Method of the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;  
using Microsoft.EntityFrameworkCore;  
namespace EFCoreCodeFirstDemo  
{  
    public class Program  
    {  
        static async Task Main(string[] args)  
        {  
            try  
            {  
                //Creating the Disconnected Entity Graph  
                //Student Entity Graph (Student plus Standard, and  
StudentAddress)  
                //Student is the Main Entity  
                //Standard, and StudentAddress are the Child Entities  
                var student = new Student()  
                {  
                    //Root Entity without key  
                    FirstName = "Hina",  
                    LastName = "Sharma",  
                    StandardId = 1,  
                    Standard = new Standard() //Child Entity with key value  
                    {  
                        StandardId = 1,  
                        StandardName = "STD1",  
                        Description = "STD1 Description"  
                    },  
                },  
            }  
            }  
        }  
    }  
}
```

```

StudentAddress = new StudentAddress() //Child Entity with
Empty Key
{
    Address1 = "Address Line1",
    Address2 = "Address Line2"
}
};
using var context = new EFCoreDbContext();
// Use TrackGraph to track the entire graph
context.ChangeTracker.TrackGraph(student, nodeEntry =>
{
    // Customize tracking behavior for each entity
    //Setting the Root Entity, i.e., Student
    if (nodeEntry.Entry.Entity is Student std)
    {
        if (std.StudentId > 0)
        {
            nodeEntry.Entry.State = EntityState.Modified;
        }
        else
        {
            nodeEntry.Entry.State = EntityState.Added;
        }
    }
    //Setting the Child Entity i.e. Standard and
StudentAddress
    else
    {
        if (nodeEntry.Entry.IsKeySet)
        {
            //If Key is Available set the State as Unchanged
as Per Your Requirement
            nodeEntry.Entry.State = EntityState.Unchanged;
        }
        else
        {
            // If Key is not Available set the State as Added
as Per Your Requirement
            nodeEntry.Entry.State = EntityState.Added;
        }
    }
});
//Checking the Entity State of Each Entity of student Entity
Graph
foreach (var entity in context.ChangeTracker.Entries())
{
    Console.WriteLine($"Entity:
{entity.Entry.GetType().Name}, State: {entity.State} ");
}
// Save changes to persist the changes to the database
context.SaveChanges();

```

```

        Console.WriteLine("Entity Graph Saved..");
        Console.Read();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}"); ;
    }
}
}
}

```

Output:

```

Entity: Student, State: Added
Entity: StudentAddress, State: Added
Entity: Standard, State: Unchanged
Entity Graph Saved..

```

We create a graph of entities starting with the Student entity as the root, containing related Standard and StudentAddress entities. We use the ChangeTracker TrackGraph method to track the entire entity graph. Inside the delegate provided to TrackGraph, we customize the tracking behavior for each entity based on our requirements.

When to use Disconnected Entity Graph in Entity Framework Core?

Disconnected Entity Graphs in Entity Framework Core are used in scenarios where you want to work with entity data outside of the context in which it was originally retrieved, and later, you want to reconnect those entities to a new context to perform database operations like updating, inserting, or deleting records.

This approach is common in applications where entities must be passed between different layers, such as a user interface, business logic, and data access layers, or when entities must be serialized and sent across a network. Disconnected entity graphs are useful in the following scenarios:

- **Web Applications:** In web applications, entities are often retrieved from the database within an HTTP request, but they must be passed to a different layer for processing. Later, when the HTTP request is complete, the entities may need to be updated or saved back to the database. Disconnected entities allow you to work with the data across different parts of your application.
- **Batch Processing:** In batch processing scenarios, you might retrieve many entities from the database, process them in a batch job, and then save the changes. Disconnected entities allow you to efficiently work with these entities, even if they are not continuously connected to a context.
- **Client-Server Applications:** In client-server applications, entities may be sent from a client to a server for processing and then returned to the client. Disconnected entities allow you to serialize and deserialize the data without maintaining a continuous database connection on the client side.
- **Caching:** You can use disconnected entities to store a data snapshot in memory for caching purposes. When the data is requested again, you can reattach the entities to a new context to apply updates or insert new data.

In the next article, I will discuss Working with [Stored Procedures in Entity Framework Core](#) with Examples. In this article, I try to explain **Disconnected Entity Graphs in Entity Framework Core** with Examples. I hope you enjoyed this Disconnected Entity Graph in Entity Framework Core article.